

SmartHome Gesture Control Application Report

Xzander Trevino

My Approach

The provided CNN was originally trained to recognize the hand gestures, so its final 27-class classification output does not directly correspond to the SmartHome gestures used in this project. Instead of using the CNN as the final classifier, I used it as a feature extractor.

Each video was represented by its middle frame, which was extracted from the training videos provided via part 1 of this assignment. The frame was converted to grayscale and passed through the CNN. I used the output of the model's penultimate layer as a feature vector representing the hand shape depicted via the frame. I then compare the feature vectors from the test videos with the feature vectors from the labeled training videos using cosine similarity in my main file.

My Solution

For every training and test video, I extracted the middle frame using the provided frame extractor function. The extracted image was converted to grayscale before being passed to the hand-shape feature extractor.

The CNN architecture contains a 64 value dense layer followed by a final 27 value output layer. To obtain the penultimate layer representation, I rebuilt the model using the existing CNN layers while excluding the final classification layer. This produced a 64 value feature vector for each gesture frame.

For each test video, I calculated cosine similarity between its feature vector and every training feature vector. The training vector with the highest cosine similarity was selected as the closest match, and its associated gesture label became the prediction for the test video.

The predicted gesture names were mapped to the required numeric labels from 0–16 and written to the defined csv file. The file contains one integer per row per label / video.

Evaluation of Solution

I evaluated the predictions locally by comparing them with the gesture names contained in the provided test filenames. In my final test run, the application correctly recognized 3 of the 51 test videos, giving an accuracy of approximately 5.9%. For comparison, random guessing across 17 equally likely gesture classes would also have an expected accuracy of approximately 5.9%.

Most of the predictions were concentrated in a small number of gesture classes, particularly LightOff, Num0, and FanDown from the ones that I observed in the output. This indicates that the extracted feature vectors did not separate the SmartHome gestures effectively.

My initial assumption to this is possibly the quality of my recording training videos. Perhaps too much background noise or not clear enough gestures displayed. The differences between the gestures used to train the model versus the smarthome gestures used may have had some variance in different attributes (hand position, orientation, background, etc.).